

- Distributed Hash Tables (DHT)
 - Complete Study Guide
 -  Table of Contents
 - 1. The Centralization Problem
 - The Scenario
 - Why Centralization Fails
 - What DHTs Do Instead
 - 2. Hashing Fundamentals — The pos() Function
 - The Code
 - How ord() Works
 - Full Step-by-Step Execution of Every Cell
 - The 100-Slot Address Ring
 - Why This Hash Is Weak
 - 3. The Collision Problem
 - What Is a Collision?
 - Why Collisions Are Mathematically Inevitable
 - The Lookup Cost
 - 4. Collision Resolution Strategies
 - Strategy 1 — Linear Probing: "First Empty Next Cell"
 - Strategy 2 — Exponential Probing: "First Empty Next Exponential Cell"
 - Strategy 3 — Chaining: "Linked List Storage"
 - Visual Summary (directly from your 1.png notes)
 - 5. Better Hashing — MD5
 - The Problem pos() Cannot Solve
 - MD5 Fixes This
 - Why MD5 Is Fundamentally Better
 - The Avalanche Effect
 - 6. Address Space & Scalability
 - How Big Is a 4-Byte Address Space?
 - "How Long to Scan Every Address?"
 - Why This Scales Catastrophically
 - 7. The $O(\log n)$ Insight
 - The Calculation
 - What This Means in Practice
 - The Binary Search Mental Model
 - 8. DHT Architecture — Core Concepts
 - The Three Pillars (from your notes)

- [The Lookup Chain](#)
- [Identity Model](#)
- [File Storage Model](#)
- [Full Architecture Picture](#)
- 9. Chord DHT
 - [9.1 The Ring](#)
 - [9.2 The Successor Rule](#)
 - [9.3 Why Naive Routing Fails](#)
 - [9.4 Finger Tables — The Fix](#)
 - [9.5 Lookup Walkthrough — lookup\(54\)](#)
 - [9.6 Chord Complexity Summary](#)
- 10. Kademlia DHT
 - [10.1 The Core Idea: XOR Distance](#)
 - [10.2 The Binary Trie](#)
 - [10.3 Building a Routing Table — Node 111's View](#)
 - [10.4 The k-Bucket Parameter](#)
 - [10.5 Why XOR Is Elegant](#)
 - [10.6 Kademlia Lookup](#)
- 11. Chord vs Kademlia
 - [Mental Models](#)
- [Quick Reference — Full DHT Lifecycle](#)

Distributed Hash Tables (DHT)

Complete Study Guide



Table of Contents

#	Section	Core Idea
1	The Centralization Problem	Why a single directory server is dangerous
2	Hashing Fundamentals	The <code>pos()</code> function, ASCII math, address rings
3	The Collision Problem	Why "Ali" and "Ail" break naive hashing

#	Section	Core Idea
4	Collision Resolution	Linear probing, exponential probing, chaining
5	Better Hashing — MD5	How cryptographic hashes fix collisions
6	Address Space & Scalability	256^4 addresses, scanning cost, why brute force dies
7	The $O(\log n)$ Insight	$\log_2(4 \text{ billion}) = 32$ — the entire reason DHTs work
8	DHT Architecture	Predictable address, routing tables, identity model
9	Chord DHT	Ring, finger tables, lookup walkthrough
10	Kademlia DHT	XOR distance, binary trie, k-buckets
11	Chord vs Kademlia	Side-by-side comparison

1. The Centralization Problem

Source: your notes (2.png)

The Scenario

You want to download a movie. In a centralized world, the address looks like this:

```
Address
  Server → tinymovies.com
  Path   → /film/batman/batman1.mkv

Protocol → TCP, UDP, HTTP, gRPC, ...
```

The server **tinymovies.com** is both the **storage** and the **directory**. You ask it where things are; it tells you. Simple.

But now imagine the content is actually spread across **many independent servers**, and there is a **Directory Service** that acts as a central index:

```
Directory Service → tinymovies.com/list      (like a Search Engine)
|
├── ID 10 → 10.10.10.1/list
├── ID 5  → 10.10.10.5/list
└── ID 20 → 10.10.10.20/list
```

The numbers **10**, **5**, **20** on the left are **node IDs** — each server has a logical identifier that determines which files it is responsible for.

Why Centralization Fails

Problem	What Happens
Single point of failure	tinymovies.com goes down → entire network is dead
Bottleneck	Every request funnels through one server → slow under load
Censorship target	Shut down the directory → shut down everything
Trust	You must trust the directory to be honest

What DHTs Do Instead

DHTs **eliminate the central directory entirely**. Every node is simultaneously:

- A **storage node** (holds some files)
- A **routing node** (helps others find files)
- A **directory fragment** (knows about nearby nodes)

The key question DHTs answer: *"Given a file name, how do I find which of millions of nodes holds it — without asking any central authority?"*

2. Hashing Fundamentals — The **pos()** Function

The Code

```
def pos(k):
    chars = list(k)           # Split string into individual characters
    s = 0                     # Running sum
    for c in chars:
        s += ord(c)           # ord() converts character → its ASCII integer
    s = s % 100               # Modulo 100 maps result into range [0, 99]
    return s
```

This is a **naive hash function**. It takes any string and maps it to a single number in `[0, 99]`. That number is a **position** on an address space — imagine a circle with 100 slots.

How `ord()` Works

Every character has a unique integer in the ASCII table. The ones relevant here:

Uppercase	Lowercase
A → 65	a → 97
M → 77	e → 101
R → 82	h → 104
S → 83	i → 105
	l → 108
	m → 109
	r → 114
	y → 121
	z → 122

Full Step-by-Step Execution of Every Cell

```
pos("Ali")
A = 65, l = 108, i = 105
sum = 65 + 108 + 105 = 278
278 % 100 = 78      ✓ output: 78

pos("Reza")
R = 82, e = 101, z = 122, a = 97
```

```
sum = 82 + 101 + 122 + 97 = 402
402 % 100 = 2          ✓ output: 2
```

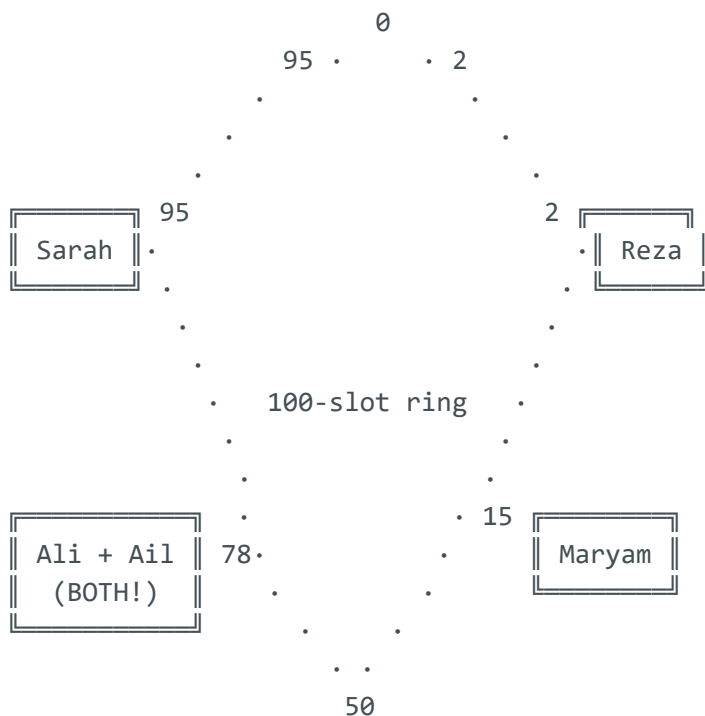
```
pos("Maryam")
M = 77, a = 97, r = 114, y = 121, a = 97, m = 109
sum = 77 + 97 + 114 + 121 + 97 + 109 = 615
615 % 100 = 15        ✓ output: 15
```

```
pos("Sarah")
S = 83, a = 97, r = 114, a = 97, h = 104
sum = 83 + 97 + 114 + 97 + 104 = 495
495 % 100 = 95        ✓ output: 95
```

```
pos("Ail")                                ⚠ COLLISION
A = 65, i = 105, l = 108
sum = 65 + 105 + 108 = 278      ← identical sum to "Ali"!
278 % 100 = 78                  ✓ output: 78  ← same slot as "Ali"!
```

The 100-Slot Address Ring

All five names mapped onto a circle:



Why This Hash Is Weak

Addition is **commutative** ($a + b = b + a$), so character ORDER doesn't matter:

```
"Ali" → 65 + 108 + 105 = 278 → 78
"Ail" → 65 + 105 + 108 = 278 → 78 ← SAME!
```

Any anagram of the same letters produces the exact same hash.
"listen" and "silent" would collide.
"astronomer" and "moon starer" would collide.

This is the **fundamental flaw** — the hash function has no sensitivity to character position.

3. The Collision Problem

Source: DHT.ipynb cells 1–5, notes from 1.png

What Is a Collision?

Two different keys hash to the **same position**. Both "Ali" and "Ail" want slot 78:

```
Index:    ... 76   77   78   79   80   ...
           ↑↑
        Ali └┐ Ail
        Both want this single slot!
```

Why Collisions Are Mathematically Inevitable

This is the **Pigeonhole Principle**:

```
Input space:  INFINITE  (any string of any length)
Output space: FINITE    (0 to 99 here; 0 to 2^n-1 in general)

∞ inputs → finite slots = collisions MUST happen
```

No hash function can avoid collisions entirely. The goals are:

1. **Minimize** collisions (use a good hash)
2. **Handle** them when they occur (collision resolution)

The Lookup Cost

From your notes (1.png), the lookup chain is:

```
ali → position? → f(x) → 20 → O(k)
```

Meaning:

"ali"	: the key we're searching for
position?	: "where should this be?"
f(x)	: the hash function computes the position
20	: the resulting slot index
O(k)	: lookup cost is O(k) where k = number of items at that slot

If the slot has no collision, $k = 1 \rightarrow O(1)$. If 5 items collide, $k = 5 \rightarrow O(5)$. Without a hash function at all, you'd do **linear search O(n)** across the entire table — which is exactly what your notes state at the top of 1.png:

```
Search → Linear Search O(n)
```

```
NID, Name, Family, Credit    ← these are fields in a database record.  
                             Without hashing, finding "ali" means  
                             scanning every single record: O(n).
```

The hash function reduces $O(n)$ to $O(k)$ where k is typically 1 or very small.

4. Collision Resolution Strategies

Source: DHT.ipynb comment cell + 1.png diagrams

Your notebook comment states the three strategies:

```
# if contents be the in same address, we can use hashing functions  
# like md5 to generate unique addresses and if still be same we can do these:  
#  
# 1. First Empty Next Cell  
# 2. First Empty Next Exponential Cell  
# 3. Linked List Storage
```


Strategy 1 — Linear Probing: "First Empty Next Cell"

If the target slot is taken, walk forward one cell at a time until you find an empty one.

Collision at slot 7. Items x and y both hash here.

BEFORE (x is already placed):

[]	[]	[]	[]	[x]	[]	[]	[]	[]	[]	[]
0	1	2	3	4	5	6	7	8	9	...

INSERT y → slot 7 is full → try 8 → empty ✓

AFTER:

[]	[]	[]	[]	[x]	[y]	[]	[]	[]	[]	[]
0	1	2	3	4	5	6	7	8	9	...

↑

↑

original overflow placed here (first empty next cell)

Problem: Creates **clusters** — long consecutive runs of filled slots that slow down future lookups.

Strategy 2 — Exponential Probing: "First Empty Next Exponential Cell"

Instead of +1, +1, +1... jump by **powers of 2**: +1, +2, +4, +8, +16...

Collision at slot 2. Probe sequence: 2, 3, 4, 6, 10, ...

+0	+1	+2	+4	+8							

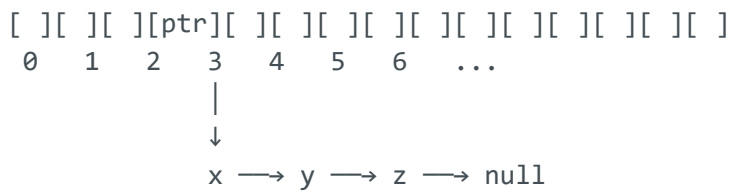
[]	[x]	[]	[x]	[]	[x]	[]	[]	[]	[]	[x]	[]	[]
0	1	2	3	4	5	6	7	8	9	10	11	
	↑		↑		↑					↑		
	+1		+2		+4					+8		

Probes spread across the table → avoids clustering.

Each step doubles the jump → logarithmic probing behavior.

Strategy 3 — Chaining: "Linked List Storage"

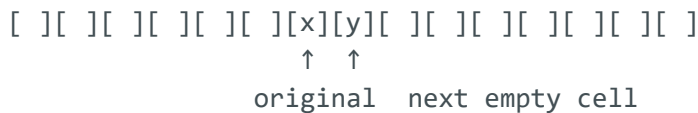
Each slot holds a **pointer to a linked list** of all items that hash there.



- **No clustering** — collisions at one slot don't affect any other slot
- **Insert:** $O(1)$ — prepend to the list
- **Lookup:** $O(k)$ — walk the chain where k = chain length
- **Trade-off:** Extra memory for pointers

Visual Summary (directly from your 1.png notes)

LINEAR PROBING:



EXPONENTIAL PROBING:



CHAINING (linked list):



5. Better Hashing — MD5

Source: DHT.ipynb cells 6–8

The Problem `pos()` Cannot Solve

```
pos("Ali") = 78
pos("Ail") = 78 ← collision. Same letters, different order, same hash.
```

No amount of collision resolution fixes the **root cause**: the hash function itself is blind to character order.

MD5 Fixes This

Your notebook runs these shell commands:

```
echo -n Ali | md5sum
# → 7a9b46ab6d983a85dd4d9a1aa64a3945

echo -n Ail | md5sum
# → c4a2c2078019464e402883f603a2b70f
```

Completely different 128-bit outputs for inputs that differ only in letter order.

Why MD5 Is Fundamentally Better

Property	<code>pos()</code> (sum)	MD5
Output size	7 bits (0–99)	128 bits (3.4×10^{38} values)
Order-sensitive	❌ No	✅ Yes
Avalanche effect	❌ No	✅ Yes
Collision probability	~1% with 10 keys	~1 in 10^{38}

The Avalanche Effect

A tiny change in input causes a **massive, unpredictable** change in output:

```
"Ali" → 7a9b46ab 6d983a85 dd4d9a1a a64a3945
"Ail" → c4a2c207 8019464e 402883f6 03a2b70f
      ^^^^^^^^^ ^^^^^^^^^ ^^^^^^^^^ ^^^^^^^^^
```

Almost every byte changed from just swapping two letters!

Note: MD5 is no longer cryptographically secure (collision attacks exist).

Production DHTs use **SHA-1** (160-bit, used in Kademlia) or **SHA-256**. For DHT purposes, what matters most is **uniform distribution** and **avalanche effect**, not resistance to adversarial attacks.

6. Address Space & Scalability

Source: *DHT.ipynb* cell 9

How Big Is a 4-Byte Address Space?

```
import math

math.pow(256, 4)    # → 4,294,967,296  (≈ 4.3 billion)
math.pow(254, 4)    # → 4,162,314,256  (≈ 4.2 billion)
```

Each byte has 256 possible values (0–255). Four bytes give you:

```
Address:  [ byte0 ][ byte1 ][ byte2 ][ byte3 ]
Values:    0–255   0–255   0–255   0–255

Total slots = 256 × 256 × 256 × 256
            = 2564
            = 4,294,967,296
```

This is exactly the **IPv4 address space** — every possible IP like **10.10.10.1**.

The second calculation **254⁴** shows what happens if you reserve 2 values per byte (e.g., network/broadcast addresses) — you lose ~130 million addresses, but still have over 4 billion.

"How Long to Scan Every Address?"

```
print(1 * 4_000_000_000 * 5)    # → 20,000,000,000
# Comment: "time to scan all addresses at 5gb/s"
```

The calculation models **brute-force network scanning**:

$$4 \times 10^9 \text{ addresses} \times 5 \text{ ns per address} = 20 \times 10^9 \text{ ns} = 20 \text{ seconds}$$

At ~5 Gbps effective throughput, pinging every single IPv4 address takes roughly 20 seconds.

Why This Scales Catastrophically

20 seconds sounds manageable for 32 bits. But DHTs use much larger address spaces:

Address bits	Total addresses	Time to brute-force scan
32 bits	4.3×10^9	~20 seconds
160 bits	1.5×10^{48}	~ 10^{39} years
256 bits squared	1.2×10^{77}	longer than the age of the universe,

Kademlia uses 160-bit IDs (SHA-1).
Brute-force search is physically impossible.
You NEED an efficient routing algorithm.

7. The O(log n) Insight

Source: DHT.ipynb cell 12

The Calculation

```
math.log2(4_000_000_000)  # → 31.897...  
  
# "With 32 bit address space we can have 4 billion addresses  
# and with log2 algorithm we can find each address in maximum 32 steps"
```

What This Means in Practice

Strategy	Steps to find 1 key among 4 billion nodes	
Linear scan	4,000,000,000 steps	$\leftarrow O(n)$
DHT routing	32 steps	$\leftarrow O(\log_2 n)$

The Binary Search Mental Model

4 billion entries. You want to find one.

LINEAR SEARCH:

[→][→][→][→][→][→][→][→]... 4 billion checks ...→[TARGET]

BINARY / DHT ROUTING:

Step 1: Is target in first half or second half?	Discard half.	2B remain.
Step 2: First half or second half?	Discard half.	1B remain.
Step 3: ...		500M remain.
...		
Step 32: Found it.		1 remain. ✓

Each hop cuts the remaining search space in HALF.
 $\log_2(4,000,000,000) \approx 32$ hops maximum. Ever.

This is the **entire mathematical foundation** of DHTs. Both Chord and Kademlia achieve $O(\log n)$ routing by implementing this binary-halving strategy across a distributed network.

8. DHT Architecture — Core Concepts

Source: 3.png + DHT.ipynb comments

The Three Pillars (from your notes)

Predictable Address	→ You can COMPUTE where something should be
Routing Table	→ Each node knows HOW to reach others
id → IP	→ The table maps logical IDs to real network addresses

The Lookup Chain

Your notes show this chain labeled **(Chord)**:

Alg → FileID → ID

Expanded:

1. You have a filename: "batman1.mkv"
2. Hash it to get a FileID: `hash("batman1.mkv") = 54`
3. FileID determines a node ID: key 54 belongs to node N56
4. Route there using the Alg: Chord routes you to N56 in $O(\log n)$ hops
5. N56 serves you the file.

Identity Model

Identity → (id, IP)

Every node has exactly two things:

- id : a hash-derived logical identifier (e.g., SHA-1 of its public key)
- IP : its real network address (how to actually reach it)

Every node supports three operations:

- join : "I'm new. Here's my id. Give me my routing table."
- leave : "I'm leaving. Here are my files. Update your tables."
- update : "My info changed. Here's the new version."

File Storage Model

FileID → (id, FileContent)

Every file stored in the DHT has:

- id : hash of the file/filename → determines WHERE it's stored
- FileContent : the actual bytes of the file

Rule: the node RESPONSIBLE for a file is the one whose id is "closest" to the file's id (definition of "closest" differs between Chord and Kademlia).

Full Architecture Picture



Node A
id = 10, IP = 10.10.10.1
Routing Table: { 20 → .10.5,
 42 → .10.20 }
Stores files with hash in [10,20)

↓ (routing)

Node B
id = 20, IP = 10.10.10.5
Routing Table: { 42 → .10.20,
 10 → .10.1 }
Stores files with hash in [20,42)

↓ (routing)

Node C
id = 42, IP = 10.10.10.20
Routing Table: { 10 → .10.1,
 20 → .10.5 }
Stores files with hash in [42,10)

No central server. Any node can find any file.

9. Chord DHT

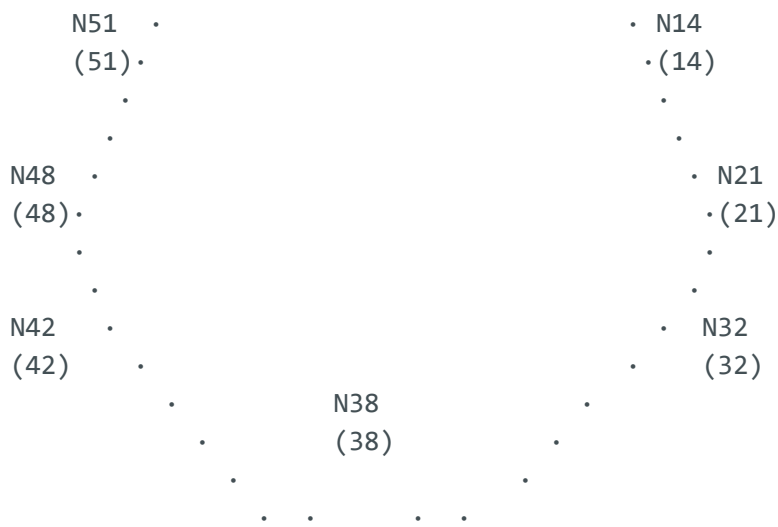
Source: chord.png (both diagrams) + 3.png + DHT.ipynb

9.1 The Ring

Chord arranges all nodes on a **logical circle**. Each node has an ID in $[0, 2^m)$ where m is the bit-width. Nodes sit at their ID position on the ring.

The diagram shows a ring with $m = 6$ (IDs 0–63) and these active nodes:





9.2 The Successor Rule

A **key** with ID **k** is stored on the **first node with ID $\geq k$** going clockwise. That node is called **k's successor**.

Where is key K54 stored?

Walking clockwise from 54:

N42 $\rightarrow 42 < 54$ X (too small)

N48 $\rightarrow 48 < 54$ X

N51 $\rightarrow 51 < 54$ X

N56 $\rightarrow 56 \geq 54$ ✓ $\leftarrow$ K54 is stored at N56

9.3 Why Naive Routing Fails

If each node only knew its **immediate successor** (next clockwise), lookup = walking the ring one node at a time:

Naive lookup for key 54, starting at N8:

N8 $\rightarrow$ N14 $\rightarrow$ N21 $\rightarrow$ N32 $\rightarrow$ N38 $\rightarrow$ N42 $\rightarrow$ N48 $\rightarrow$ N51 $\rightarrow$ N56

8 hops for a 9-node ring.

For 1 million nodes: up to 1 million hops. $O(n)$.

9.4 Finger Tables — The Fix

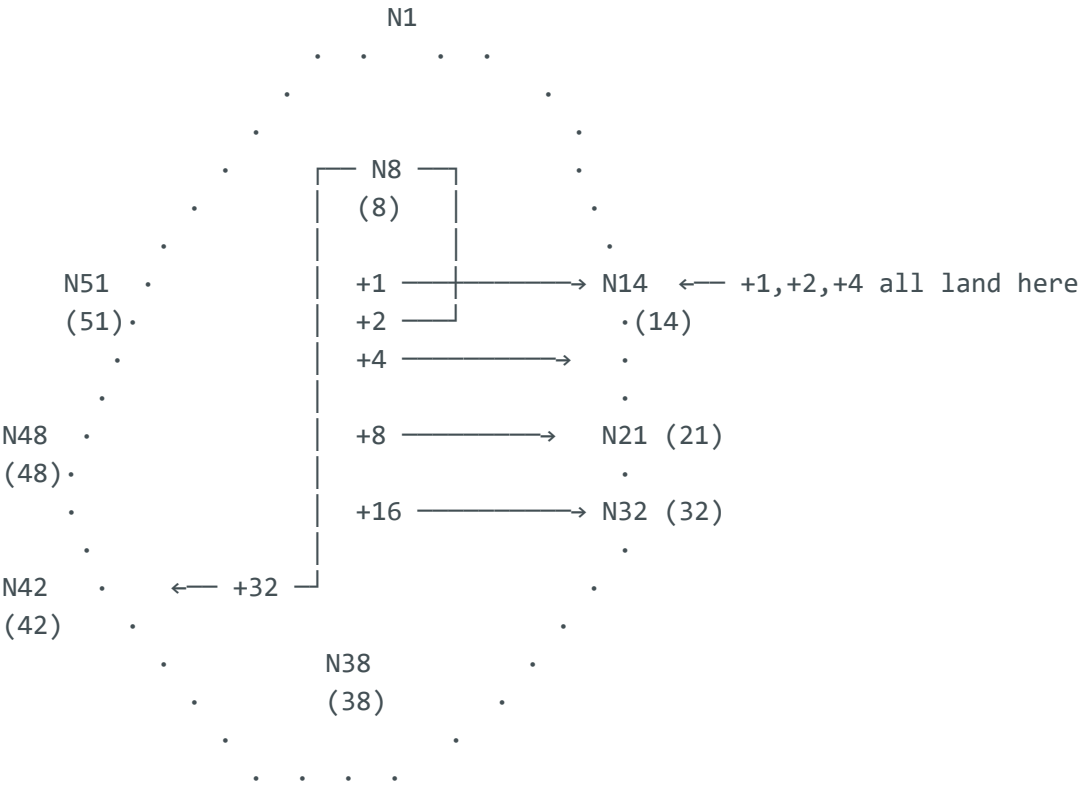
Each node stores a **finger table** with m entries. Finger i points to the successor of $(node_id + 2^i) \bmod 2^m$. This creates **shortcut pointers** that jump exponentially farther around the ring.

N8's Finger Table (from the diagram, $m = 6$):

Finger i	Start = $N8 + 2^i$	Successor	Explanation
0	$8 + 1 = 9$	N14	First node at or after 9
1	$8 + 2 = 10$	N14	First node at or after 10
2	$8 + 4 = 12$	N14	First node at or after 12
3	$8 + 8 = 16$	N21	First node at or after 16
4	$8 + 16 = 24$	N32	First node at or after 24
5	$8 + 32 = 40$	N42	First node at or after 40

N8 can jump DIRECTLY to: N14, N21, N32, N42
(skipping all intermediate nodes)

Visualized on the ring — the shortcut arrows from the left diagram:



9.5 Lookup Walkthrough — $lookup(54)$

Goal: Starting at N8, find which node stores key 54.

Algorithm at each step: Look at your finger table. Find the **largest finger that is less than the target** (closest predecessor). Forward the query there.

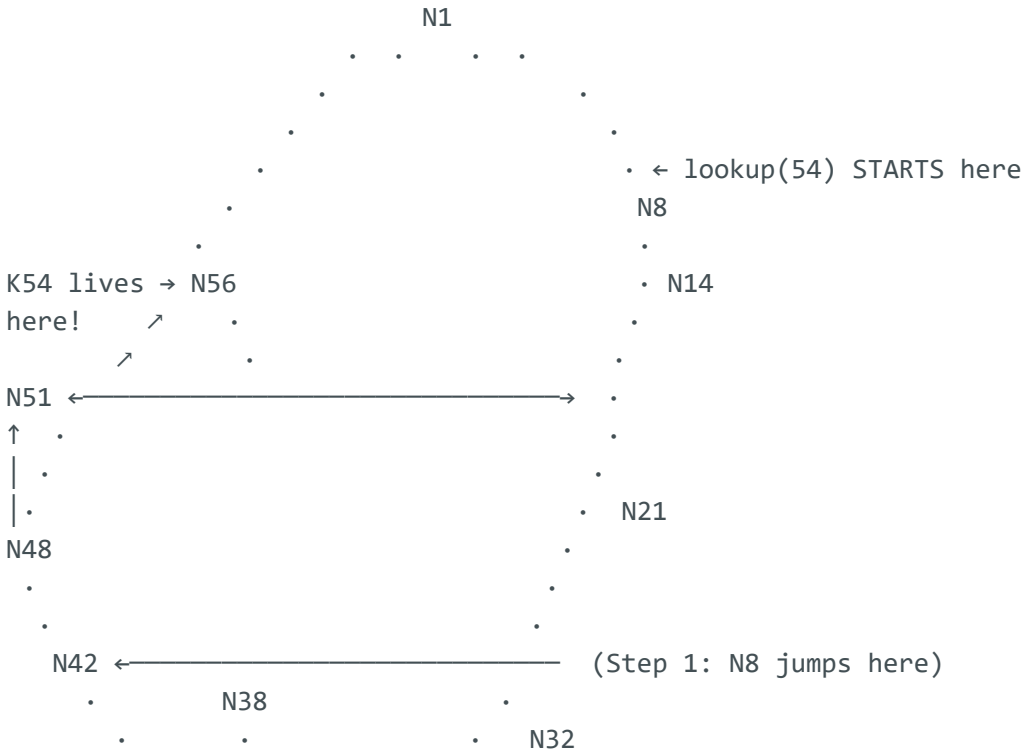
```
Step 1: N8 receives lookup(54)
        Finger table entries: N14, N21, N32, N42
        Which is the largest entry < 54?
            N14 < 54 ✓
            N21 < 54 ✓
            N32 < 54 ✓
            N42 < 54 ✓ ← largest!
        → Forward to N42
```

```
Step 2:  N42 receives lookup(54)
         N42's fingers include N51
         N51 < 54 ✓ ← closest predecessor
         → Forward to N51
```

```
Step 3: N51 receives lookup(54)
        N51's successor is N56
        N56 ≥ 54 ✓ ← this IS the responsible node!
        → Return: "Key 54 is at N56"
```

Result: K54 is stored at N56.
Total hops: 3 ($O(\log n)$ – not 8!)

The lookup path on the ring:



Route: N8 $\longrightarrow$ N42 $\longrightarrow$ N51 $\longrightarrow$ N56 (3 hops)

9.6 Chord Complexity Summary

Operation	Cost
Lookup	$O(\log n)$
Join	$O(\log^2 n)$
Leave	$O(\log^2 n)$
Finger table size per node	$O(\log n)$ entries

10. Kademlia DHT

Source: *kademlia.png* + *DHT.ipynb*

10.1 The Core Idea: XOR Distance

Chord uses a ring and measures distance by arc length. Kademlia uses a completely different metric: **XOR**.

$\text{Distance}(A, B) = A \oplus B$ (bitwise exclusive-or)

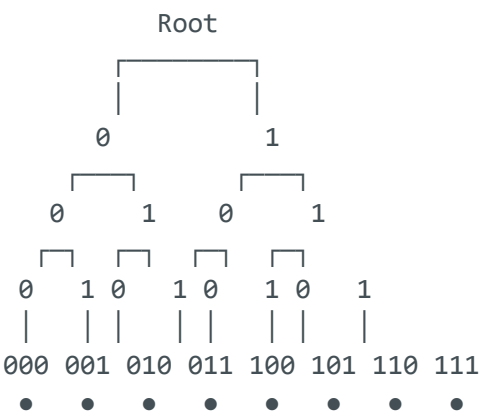
Examples with 3-bit IDs:

$\text{Distance}(000, 001) = 001 = 1$	$\leftarrow$ very close (differ in last bit only)
$\text{Distance}(000, 010) = 010 = 2$	
$\text{Distance}(000, 100) = 100 = 4$	$\leftarrow$ far (differ in first/most-significant bit)
$\text{Distance}(111, 110) = 001 = 1$	$\leftarrow$ very close
$\text{Distance}(111, 000) = 111 = 7$	$\leftarrow$ maximum distance (all bits differ)

Key insight: Nodes that share more **leading bits** in their ID are "closer" in XOR distance. This naturally creates a **binary tree** structure.

10.2 The Binary Trie

All possible node IDs can be organized into a binary tree (trie). Each leaf is one ID. The tree reveals distance relationships visually:



Nodes sharing a longer common prefix are CLOSER together:

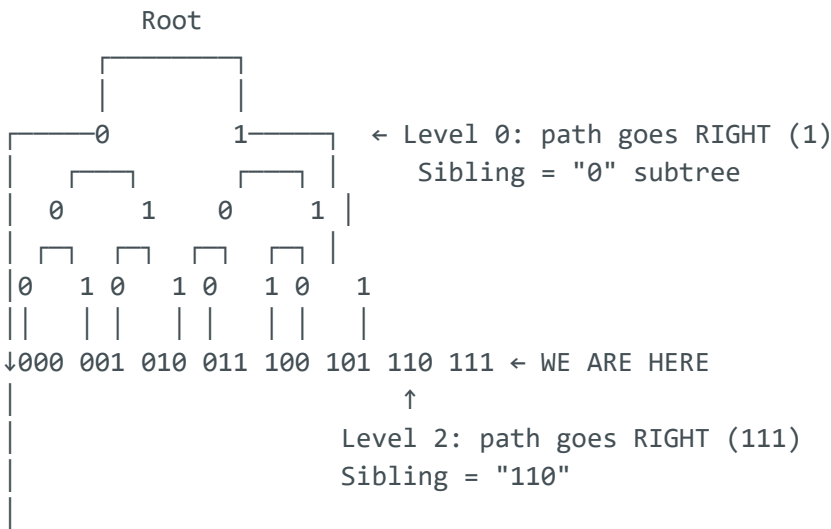
- 111 and 110 → share prefix "11" → distance = 001 = 1 (close)
- 111 and 100 → share prefix "1" → distance = 011 = 3 (medium)
- 111 and 000 → share prefix "" → distance = 111 = 7 (far)

10.3 Building a Routing Table — Node 111's View

This is exactly what the kademia.png diagram shows.

Node **111** builds its routing table by tracing its path from root to leaf. At **each level**, it stores info about the **sibling subtree** — the branch it did NOT take:

Path of node 111 through the trie:



Level 0 sibling subtree
(all IDs starting with 0)

The three k-buckets node 111 maintains:

k-bucket	Contents	Why
for "0"	{ 000 }	Sibling at level 0 (root). The "0" subtree is the FARTHEST region. We store one representative: node 000.
for "10"	{ } (empty)	Sibling at level 1. "10" is not a prefix of any active node, so this bucket is empty.
for "110"	{ 110 }	Sibling at level 2 (leaf level). Node 110 is our NEAREST neighbor. (XOR distance = $111 \oplus 110 = 001 = 1$)

10.4 The k-Bucket Parameter

Each entry in the routing table is called a **k-bucket**. The **k** parameter limits how many nodes each bucket can hold:

- k = 1: Each bucket holds at most 1 node.
Simple, but if that node goes offline, the bucket is empty.
(This is what the diagram shows.)
- k = 20: Each bucket holds up to 20 nodes.
Redundancy – if some go offline, you still have contacts.
Also: you prefer nodes that have been online the LONGEST
(they're more likely to stay online). This is Kademlia's
natural resilience mechanism.

10.5 Why XOR Is Elegant

Property 1 – Symmetry:

$\text{Distance}(A, B) = A \oplus B = B \oplus A = \text{Distance}(B, A)$

If A thinks B is close, B also thinks A is close.

→ Both nodes naturally keep each other in their routing tables.

Property 2 – Triangle inequality:

$\text{Distance}(A, C) \leq \text{Distance}(A, B) + \text{Distance}(B, C)$

→ Routing always makes progress toward the target. No loops.

Property 3 – Zero self-distance:

$\text{Distance}(A, A) = A \oplus A = 0$

Property 4 – Unique perspective:

Each node sees a DIFFERENT distance landscape.

→ Routing tables are naturally unique per node.

→ The network self-organizes without any central coordination.

10.6 Kademlia Lookup

To find key **K**:

1. Compute target = hash(K)
2. In YOUR routing table, find the node closest to target (by XOR)
3. Ask that node: "Who do YOU know that's closest to target?"
4. It replies with nodes from its own routing table
5. Repeat with the closest node found so far
6. Stop when you reach the node responsible for target

Each round gets strictly closer (in XOR distance) to the target.

The binary trie structure guarantees $O(\log n)$ rounds.

11. Chord vs Kademlia

Feature	Chord	Kademlia
Topology	Ring (circle)	Binary trie (tree)
Distance metric	Clockwise arc length	XOR of node IDs
Routing table	Finger table: $O(\log n)$ entries	$O(\log n)$ k-buckets × k nodes each
Lookup hops	$O(\log n)$	$O(\log n)$
Symmetry	✗ Asymmetric (arc $A \rightarrow B \neq$ arc $B \rightarrow A$)	✓ Symmetric (XOR)

Feature	Chord	Kademlia
Self-organizing	Needs explicit join/stabilize protocol	Naturally self-organizes via XOR
Redundancy	Successor lists	k-buckets hold multiple nodes
Failure tolerance	Requires successor list maintenance	k-buckets naturally survive failures
Routing table updates	Active: must periodically stabilize	Passive: updated during normal lookups
Real-world usage	Academic / research systems	BitTorrent, Ethereum, I2P

Mental Models

CHORD – "Circular highway with express-lane exits" ||

You're driving around a ring.

Finger tables = highway off-ramps that let you jump ahead by 1, 2, 4, 8, 16, 32... positions.

Each jump doubles the distance. $O(\log n)$ exits to any destination.

KADEMLIA – "Binary search tree across the internet" ||

Every node sees the network as a binary tree.

Nodes that share more prefix bits are closer (by XOR).

Routing = walking down the tree toward the target.

Each hop cuts the remaining search space in half.

k-buckets give you multiple contacts at each distance level for redundancy.

Quick Reference — Full DHT Lifecycle

1. NODE JOINS

- | — id = hash(public_key)
- | — Contact a bootstrap node
- | — Learn neighbors → populate routing table
- | — "I own keys in range [my_id, successor_id)"

2. STORE FILE

- | — file_id = hash(filename)
- | — Route to responsible node ← $O(\log n)$ hops
- | — That node stores (file_id, file_content)

3. FIND FILE

- | — file_id = hash(filename) ← same hash!
- | — Route to responsible node ← $O(\log n)$ hops
- | — That node returns file_content

4. NODE LEAVES

- | — Transfer stored files to successor
- | — Notify neighbors to update routing tables
- | — Graceful exit (or crash – DHT must handle both)
